

Промежуточный отчёт по лабораторной работе №3 по предмету “Технологии управления данными”

Текст задачи:

Лабораторная работа №3. Создание витрины данных.

Витрина данных представляет собой специализированное хранилище, где данные консолидируются по определенному признаку. В данной работе требуется консолидировать результаты продаж по неделям. Это обеспечит уменьшение нагрузки на основное Хранилище данных.

Основными сущностями Витрины будут:

1. Недели;
2. Продажи.

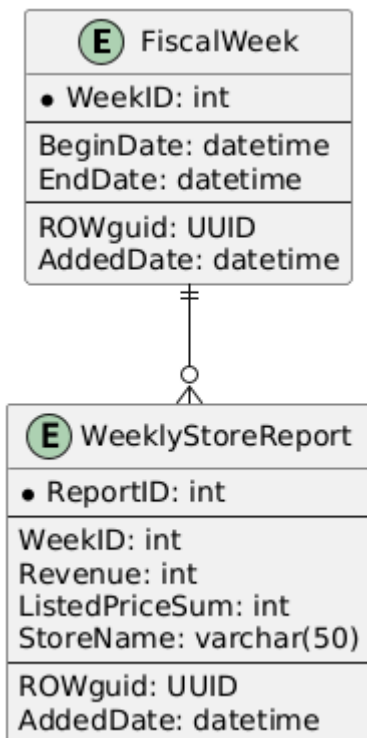
Сущность недели хранит информацию о временных промежутках, за которые была произведена консолидация данных.

Сущность Продажи обеспечивает хранение информации о результатах продаж в Филиале в соответствующую неделю.

Решение:

Выбранная реляционная СУБД - PostgreSQL.

Диаграмма связей сущностей:



FiscalWeek - сущность “Неделя”, где вся неделя хранится в виде полуинтервала [BeginDate; EndDate).

WeeklyStoreReport - сущность “Продажи” по условию.

Скрипты и код:

 entities_creation.sql

 seeding.py

entities_creation.sql - скрипт создания базы данных (в соответствии по диаграмме и описанию выше):

```
CREATE TABLE IF NOT EXISTS FiscalWeek (
    ROWguid UUID DEFAULT gen_random_uuid(),
    AddedDate TIMESTAMP DEFAULT now(),

    WeekID INTEGER PRIMARY KEY,
    BeginDate TIMESTAMP NOT NULL,
    EndDate TIMESTAMP NOT NULL
);

CREATE TABLE IF NOT EXISTS WeeklyStoreReport (
    ROWguid UUID DEFAULT gen_random_uuid(),
    AddedDate TIMESTAMP DEFAULT now(),
```

```

ReportID INTEGER PRIMARY KEY,
WeekID INTEGER NOT NULL,
StoreName VARCHAR(50) NOT NULL,
Revenue INTEGER NOT NULL,
ListedPriceSum INTEGER NOT NULL,

FOREIGN KEY (WeekID) REFERENCES FiscalWeek(WeekID)
);

```

seeding.py - скрипт консолидации данных из базы данных из лабораторной №2, написанный на языке Python:

```

import psycopg2
import datetime

def date_id(x):
    year = x.year
    month = x.month
    day = x.day
    return f'{year:04}{month:02}{day:02}'

db_config_warehouse = {
    "dbname": "warehouse_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5448"
}

db_config_mart = {
    "dbname": "mart_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5449"
}

conn_warehouse = psycopg2.connect(**db_config_warehouse)
cur_warehouse = conn_warehouse.cursor()
conn_mart = psycopg2.connect(**db_config_mart)
cur_mart = conn_mart.cursor()

start = datetime.datetime.strptime('2025-09-29 00:00', '%Y-%m-%d %H:%M')
end = datetime.datetime.strptime('2025-11-02 23:59', '%Y-%m-%d %H:%M')
monday = start
weekID = 0

```

```

reportID = 0
while monday < end:
    sunday = monday + datetime.timedelta(days=7)
    cur_mart.execute("""INSERT INTO FiscalWeek (WeekID, BeginDate, EndDate)
VALUES (%s, %s, %s)""",
                    (str(weekID), str(monday), str(sunday)))
    cur_warehouse.execute("""SELECT DISTINCT StoreID FROM SoldItem""")
    storeIDs = cur_warehouse.fetchall()
    for storeID in storeIDs:
        price = 0
        listed_price = 0
        cur_warehouse.execute("""SELECT StoreName FROM StoreInfo WHERE
StoreID = %s""",
                                (storeID,))
        store = cur_warehouse.fetchall()[0]
        cur_warehouse.execute("""
            SELECT si.ActualPrice, si.ListedPrice, si.Amount
            FROM SoldItem si
            WHERE si.DateID >= %s
            AND si.DateID < %s
            AND si.StoreID = %s""",
                                (date_id(monday), date_id(sunday), storeID))
        ...
или альтернативно:
cur_warehouse.execute("""
    SELECT si.ActualPrice, si.ListedPrice, si.Amount
    FROM SoldItem si
    JOIN Receipt r on r.ReceiptID = si.ReceiptID
    WHERE r.CompletedDate >= timestamp %s
    AND r.CompletedDate < timestamp %s
    AND si.StoreID = %s""",
                        (str(monday), str(sunday), storeID))
    ...
    rows_info = cur_warehouse.fetchall()
    for row_info in rows_info:
        price += row_info[0] * row_info[2]
        listed_price += row_info[1] * row_info[2]
    cur_mart.execute("""INSERT INTO WeeklyStoreReport (ReportID, WeekID,
StoreName, Revenue, ListedPriceSum) VALUES (%s, %s, %s, %s, %s)""",
                    (str(reportID), str(weekID), store, str(price),
str(listed_price)))
    reportID += 1
    weekID += 1
    monday += datetime.timedelta(days=7)

conn_warehouse.commit()
conn_mart.commit()
cur_warehouse.close()

```

```
cur_mart.close()
conn_warehouse.close()
conn_mart.close()
print("All done!")
```

*функция random_datetime в лабораторной №1 изменена так, чтобы размах даты был уже для более удобного тестирования информации.